

NIVEAU : MASTER 1 Génie Logiciel

COURS DOM & ÉVÉNEMENTS

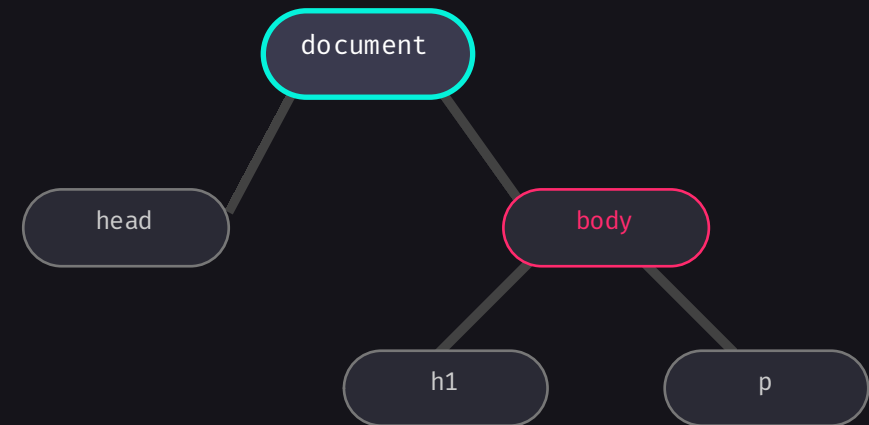
```
const element = document.querySelector('#app');  
  
element.addEventListener('click', (e) => {  
  
  e.preventDefault();  
  
  const target = e.target;  
  
  if (target.matches('.btn')) {  
  
    console.log('Action triggered');  
  
    updateDOM(target.dataset.id);  
  
  }  
});
```

Comprendre la structure arborescente, maîtriser la propagation des événements et manipuler l'interface utilisateur en temps réel.

1.1 Le DOM en Une Phrase

Le **DOM** = Une représentation **arborescente** de votre page HTML

- ◆ Chaque balise = **un nœud**
- ◆ Accessible via **JavaScript**
- ◆ Modifiable en **temps réel**



1.2 DOM vs BOM vs HTML

window (Navigateur)

BOM

navigator
location
screen
history

DOM

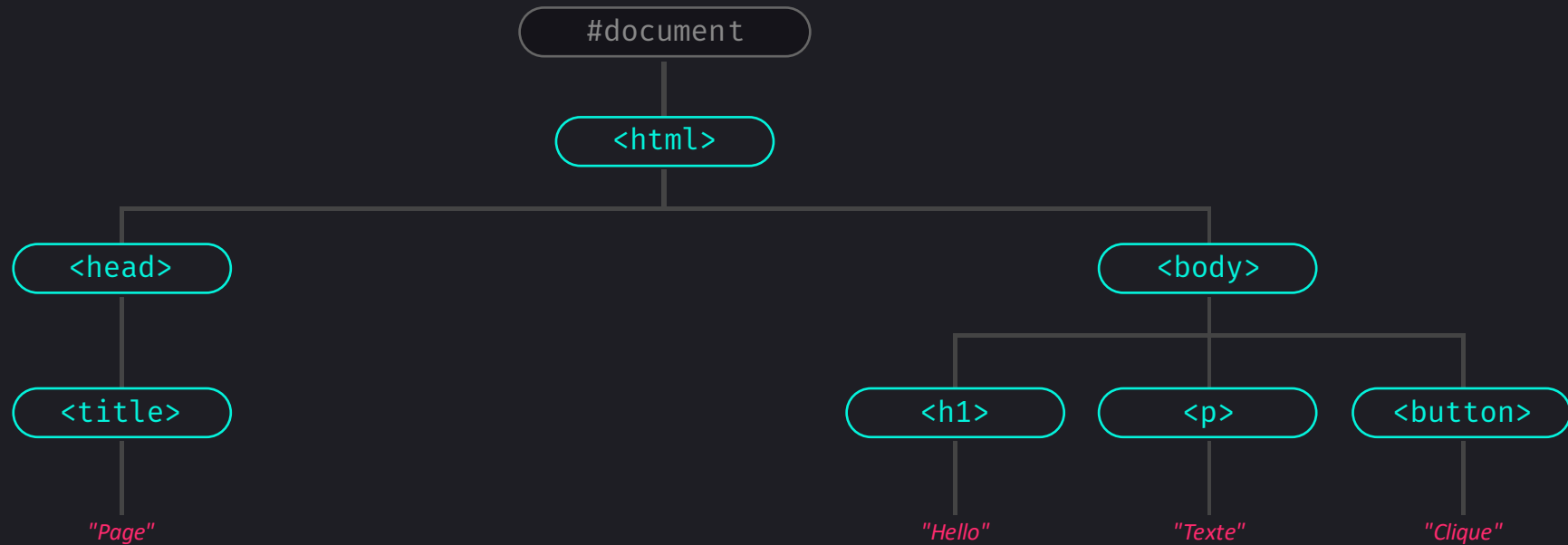
document
<html>
<body>
All Elements

JavaScript

Object
Array
Function
Votre Code

À RETENIR : DOM = contenu de la page | BOM = environnement du navigateur

1.3 Structure DOM en Arbre



Chaque élément HTML devient un nœud dans l'arbre

1.4 Types de Nœuds

TYPE	EXEMPLE	CONSTANTE JS
Element	<code><div>, <p>, <button></code>	<code>ELEMENT_NODE</code>
Text	"Bonjour", espaces, \n	<code>TEXT_NODE</code>
Document	Racine de l'arbre	<code>DOCUMENT_NODE</code>
Comment	<code><!-- commentaire --></code>	<code>COMMENT_NODE</code>



IMPORTANT : Les espaces vides et les retours à la ligne sont considérés comme des **nœuds textes** !

1.5 Accès aux Éléments Clés

Le navigateur offre des **raccourcis directs** pour accéder aux conteneurs structurels de la page la page sans passer par une recherche complexe.

- 1 Racine du document**
Accès direct à la balise `<html>`
- 2 Métadonnées**
Accès direct à `<head>`
- 3 Contenu visible**
Accès direct à `<body>`

Pourquoi utiliser ça ?

C'est plus performant et lisible que `querySelector('body')`.

```
script.js

// 1. Accès à la racine <html>
const root = document.documentElement;
// → Renvoie l'élément <html>

// 2. Accès aux métadonnées
const head = document.head;
// → Renvoie l'élément <head>

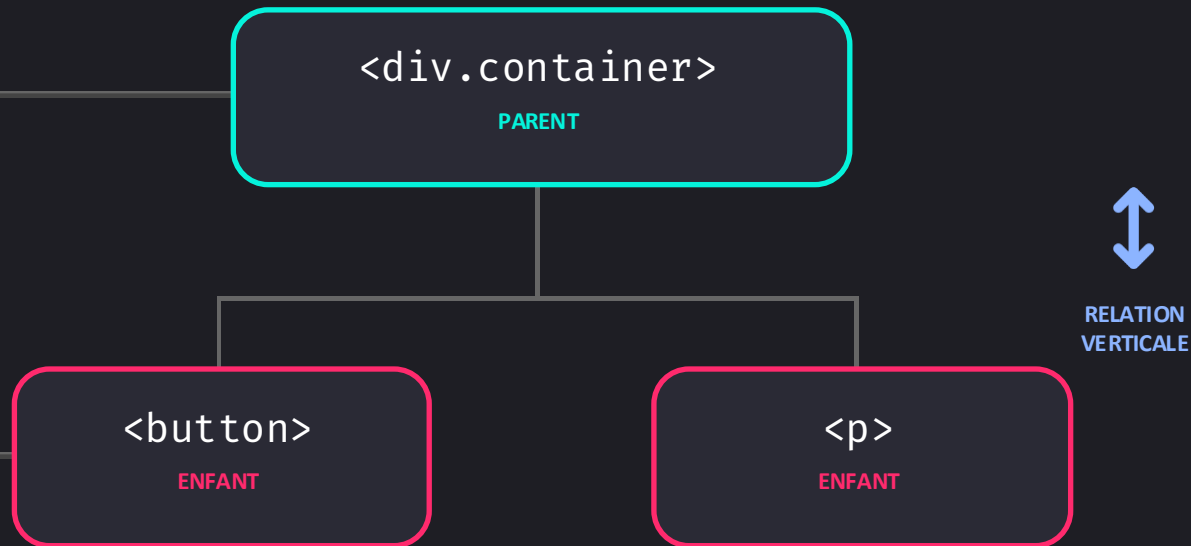
// 3. Accès au corps de page
const body = document.body;
// → Renvoie l'élément <body>

console.log("Langue :", root.lang);
```

2.1 Parents et enfants

Structure HTML :

```
<div class="container">  
<button>Clique</button>  
<p>Texte</p>  
</div>
```



À RETENIR : Naviguer verticalement dans l'arbre pour trouver Parents et Enfants

2.2 Propriétés de Navigation

Pour descendre dans l'arbre DOM depuis un parent, JavaScript offre des propriétés pour distinguer les **éléments HTML** des autres types de nœuds.

1 children

Retourne uniquement les balises HTML (ignore le texte et les espaces).

2 childNodes

Retourne TOUT : balises + texte + commentaires + retours à la ligne.

3 firstElementChild / lastElementChild

Accès direct aux extrémités (uniquement éléments).

Conseil Pro :

Privilégiez `children` dans 99% des cas pour éviter de gérer les nœuds de texte vides (espaces/retours à la ligne).

```
navigation.js

// <div class="container"> <button>...</button>
// <p>...</p> </div>
const container=document.querySelector('.container');

// 1. Uniquement les éléments (propre)
console.log(container.children);
// → HTMLCollection [button, p]

// 2. Tout le contenu (plus bruyant)
console.log(container.childNodes);
// → NodeList [text, button, text, p, text]

// 3. Accès rapide aux extrémités
const btn=container.firstElementChild; // <button>
const txt=container.lastElementChild; // <p>
```

2.3 Frères et Sœurs

Navigation **horizontale** dans l'arbre : se déplacer d'un élément à un autre de même niveau hiérarchique.

1 Parent direct

`parentNode` remonte d'un seul niveau vers le conteneur immédiat.

2 Attention aux espaces

`nextSibling` inclut les nœuds texte (retours à la ligne). Utilisez la version `"Element"`.

3 Propriétés "Element"

`nextElementSibling` saute les nœuds texte pour cibler la balise suivante.

Astuce : Privilégiez toujours les propriétés finissant par `...Element...` pour éviter les bugs liés à la mise en forme du code HTML.

```
siblings.js

const button=document.querySelector('button');

// 1. Remonter vers le parent

console.log(button.parentNode); // <div class="container">

// 2. Frère suivant (⚠ Nœud texte)

console.log(button.nextSibling); // #text (retour à la ligne)

// 3. Frère élément suivant (✅ Sûr)

console.log(button.nextElementSibling); // <p>

// 4. Frère précédent

console.log(button.previousElementSibling); // null
(aucun frère avant)
```

2.4 Collections vivantes vs Statiques

Certaines listes d'éléments se mettent à jour **automatiquement**, d'autres restent **figées**.



Collection Vivante

`getElementsBy...`

Reflète les changements du DOM en temps réel.



Collection Statique

`querySelectorAll`

Instantané (snapshot) figé au moment de l'appel.

Conseil Pro :

Préférer `querySelectorAll` (statique) car son comportement est plus prévisible et évite les boucles infinies.



console.js

```
// 1. VIVANTE (getElementsBy...)
const liveList=document.getElementsByTagName('button');
// Se met à jour toute seule

// 2. STATIQUE (querySelector...)
const staticList=document.querySelectorAll('button');
// Reste figée (snapshot)

// Modification du DOM
const btn=document.createElement('button');
document.body.appendChild(btn);

// Résultats dans la console
console.log(liveList.length); // +1 (Mis à jour !)
console.log(staticList.length); // Inchangé
```

3.1 Méthodes de sélection

JavaScript offre deux approches principales pour trouver des éléments : les méthodes **modernes** et les méthodes **historiques**.

1 Moderne & Recommandé

`querySelector` utilise la syntaxe CSS. Puissant et polyvalent.

2 Méthodes Historiques

`getElementById`, etc. Plus spécifiques et rapides, mais moins flexibles.

Conseil pro :

Privilégiez `querySelector` pour 99% des cas. C'est plus cohérent avec votre CSS et plus facile à lire.

```
selection.js

// 1. Moderne et recommandé (Syntaxe CSS)
document.querySelector('.ma-classe') // 1er élément
document.querySelectorAll('.ma-classe') // Tous

// 2. Anciennes méthodes (Encore utiles)
document.getElementById('mon-id')
document.getElementsByClassName('classe')
document.getElementsByTagName('div')

// Exemple d'utilisation
const btn=document.querySelector('#submit');
console.log(btn.textContent);
```

3.2 Exemples `querySelector`

La puissance de `querySelector` réside dans sa capacité à utiliser n'importe quel sélecteur valide.

1 Standard (ID & Classe)

Comme en CSS: `#id` et `.class`

2 ÉPÉPÉPÉ

Ciblez des types précis: `[type="email"]`

3 Relationnel

Ciblez selon la hiérarchie: `div > p`

Astuce Pro :

Si vous savez écrire le sélecteur CSS pour styliser un élément, vous savez déjà comment le sélectionner en JS !

```
selectors.js

// 1. Par classe CSS
document.querySelector('.button');

// 2. Par identifiant (ID)
document.querySelector('#login-form');

// 3. Par attribut spécifique
document.querySelector('input[type="email"]');

// 4. Sélecteur complexe (hiérarchie + pseudo-classe)
document.querySelector('div.container > button:first-child');

// 5. Sélectionner TOUS les éléments (NodeList)
const allBtns=document.querySelectorAll('button');
```

3.3 Trouver l'ancêtre (closest)

La méthode `closest()` permet de naviguer vers le haut l'arbre DOM pour trouver le parent le plus proche correspondant à un sélecteur.

1 Direction : Haut

Cherche parmi les parents (ancêtres), contrairement à `querySelector`.

2 Point de départ

Vérifie d'abord l'élément lui-même, puis ses parents.

3 Résultat unique

Retourne le premier élément trouvé ou `null`.

Cas d'usage principal :

Indispensable pour la **délégation d'événements** (voir section 6) ou pour trouver le formulaire contenant un bouton cliqué.

```
logic.js

// Imaginons un clic sur un bouton
const button=event.target;

// 1. Trouver le formulaire parent
const form=button.closest('form');

// 2. Trouver un conteneur par classe
const card=button.closest('.card');

if (form) {
    console.log('Le bouton est dans un formulaire');
}

if (!card) {
    console.log('Pas de carte parente trouvée');
}
```

4.1 Ajouter des Éléments

L'ajout de contenu dynamique se fait en deux étapes : **création** en mémoire puis **insertion** dans l'arbre.

1 Créer (Create)

`createElement()` génère un nœud orphelin (pas encore visible).

2 Configurer

Ajouter texte, classes et attributs *avant* l'insertion.

3 Insérer (Insert)

Placer l'élément dans le DOM via `appendChild`, `prepend`, etc.

Positions Clés :

`appendChild` = Fin | `prepend` = Début | `before/after` = Autour.

```
creation.js

// 1. Créer et configurer
const newDiv=document.createElement('div');
newDiv.textContent='Nouvel élément';
newDiv.classList.add('card');

// 2. Ajouter à la FIN du parent
document.body.appendChild(newDiv);

// 3. Ajouter au DÉBUT du parent
document.body.prepend(newDiv);

// 4. Ajouter RELATIVEMENT à un élément
button.before(newDiv); // Juste avant
button.after(newDiv); // Juste après
```

4.2 Insérer à une Position Précise

Pour placer un élément à un endroit spécifique spécifique (pas juste à la fin), utilisez la méthode `insertBefore`.

A Le Parent

L'élément conteneur dans lequel on insère.

B L'Élément de Référence

Le nœud existant **devant** lequel on veut insérer.

C La Syntaxe

```
parent.insertBefore(nouveau, ref)
```

Bon à savoir :

Si l'élément de référence est `null`, `insertBefore` fonctionne exactement comme `appendChild` (ajout à la fin).

```
insertion.js

// 1. Le conteneur parent
const parent=document.querySelector('.container');

// 2. L'élément de référence (cible)
const ref=parent.querySelector('p');

// 3. Le nouvel élément à créer
const newButton=document.createElement('button');
newButton.textContent='Nouveau';

// 4. INSTRUCTION CLÉ : Insérer AVANT
parent.insertBefore(newButton, ref);

/* Résultat HTML : <div class="container">
<button>Nouveau</button> ← inséré ici <p>Texte existant</p> ←
référence </div> */
```

4.3 Modifier le Texte / HTML

JavaScript offre trois propriétés principales pour lire ou modifier le contenu, chacune avec un comportement spécifique.

✓ **textContent**

Sûr et rapide. N'interprète pas les balises HTML. À privilégier.

⚠ **innerHTML**

Parse le HTML. Utile pour des structures complexes, mais risqué (XSS).
risqué (XSS).

⚡ **outerHTML**

Remplace l'élément lui-même (balises incluses) dans le DOM.

⚠ **innerHTML**

Utilisez toujours `textContent` par défaut, sauf si vous avez explicitement besoin d'insérer du HTML.

```
content.js

const title=document.querySelector('h1');

// 1. Sûr : Texte brut uniquement
title.textContent='Bonjour';
// Résultat : <h1>Bonjour</h1>

// 2. HTML : Interprète les balises
title.innerHTML='<span>Bonjour</span>';
// Résultat : <h1><span>Bonjour</span></h1>

// 3. Remplace l'élément lui-même
title.outerHTML='<h2>Bonjour</h2>';
// Résultat : <h2>Bonjour</h2> (le h1 disparaît)
```

4.4 Attention à innerHTML

L' utilisation imprudente de `innerHTML` est la porte ouverte aux attaques XSS (Cross-Site Scripting).



Interprétation du code

Le navigateur exécute tout script contenu dans la chaîne HTML (ex: HTML (ex: balises `<script>` ou attributs d'événements)).



Solution sécurisée

Utilisez toujours `textContent` ou `innerText` pour insérer des données provenant de l'utilisateur.

Règle d'or :

Ne jamais utiliser `innerHTML` avec des données que vous ne contrôlez pas à 100% (saisie utilisateur, URL, API tierce).



security.js

// Simulation d'une saisie malveillante

```
const userInput='<img src=x
onerror="alert(\'Piratée!\')">';
```

//  **DANGEREUX : Injection XSS**

```
document.querySelector('.comment').innerHTML=userInput;
// → Le navigateur exécute l'alerte JS !
```

//  **SÛR : Traité comme du texte**

```
document.querySelector('.comment').textContent=userInput;
// → Affiche littéralement : <img src=x ...>
// → Aucun code n'est exécuté.
```

4.5 Modifier les Classes

L'API **classList** est l'approche moderne manipuler les classes CSS de manière propre.



.add()

Écriture d'une classe



.remove()

Retire une classe existante



.toggle()

Ajoute si absent, retire si présent



.contains()

Vérifie la présence (booléen)

Conseil :

Préférez `classList` à `className` car il permet de gérer plusieurs classes sans écraser les autres.



style-logic.js

```
const button=document.querySelector('button');

// 1. Ajouter une classe
button.classList.add('active');

// 2. Retirer une classe
button.classList.remove('active');

// 3. Basculer (Ajouter/Retirer)
button.classList.toggle('active');

// 4. Vérifier la présence
if (button.classList.contains('active')) {
  console.log('Bouton actif');
}
```

4.6 Supprimer des Éléments

Pour retirer des nœuds du DOM, vous avez le choix le choix entre une méthode **moderne directe** et une et une méthode **historique**.

1 Méthode moderne : `.remove()`

On appelle la méthode directement sur l'élément à supprimer. Simple et lisible.

2 `parentNode.removeChild()`

Nécessite d'accéder d'abord au parent, puis de lui demander de retirer de retirer l'enfant.

Recommandation :

Privilégiez toujours `element.remove()` pour sa clarté, sauf si vous devez supporter Internet Explorer (très rare).

```
script.js

// 1. Moderne : Supression directe

const element=document.querySelector('.old-card');

element.remove(); // ← Clair et net

// 2. Ancienne méthode (Verbeuse)

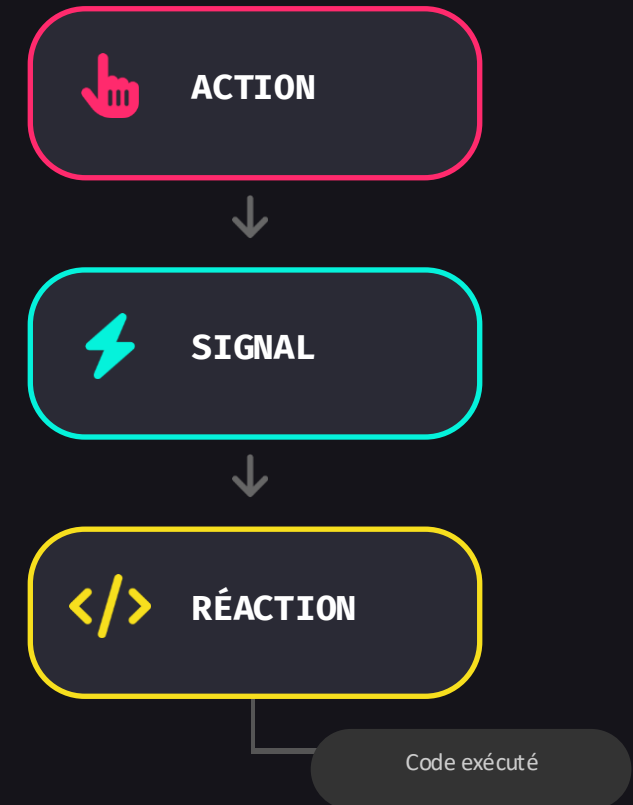
const parent=document.querySelector('.container');
const child=parent.querySelector('p');

// Il faut demander au parent !
parent.removeChild(child);
```

5.1 Qu'est-ce qu'un Événement ?

Utilisateur fait une **action**
→ Navigateur envoie un **signal**
→ Code JavaScript **réagit**

- ◆ Événement = **notification** du navigateur
- ◆ Interactions : **clic**, **toucher**, **saisie**
- ◆ Système : **soumission**, **scroll**, **resize**



5.2 Événements Courants

Catégorie	Liste des Événements
Souris	<code>click</code> , <code>dblclick</code> , <code>mouseover</code> , <code>mouseout</code> , <code>mousemove</code>
Clavier	<code>keydown</code> , <code>keyup</code> , <code>keypress</code>
Formulaire	<code>submit</code> , <code>input</code> , <code>change</code> , <code>focus</code> , <code>blur</code>
Document	<code>DOMContentLoaded</code> , <code>load</code> , <code>resize</code> , <code>scroll</code>



LES INCONTOURNABLES : Les 3 événements les plus utilisés sont `click`, `input` et `submit`.

5.3 Attacher un Écouteur

La méthode `addEventListener()` est le standard pour réagir aux interactions sans modifier le HTML.

- 1 L'Élément cible**
On sélectionne l'objet du DOM (ex: bouton).
- 2 L'Événement**
Une chaîne de caractères (ex: `'click'`).
- 3 Le Listener (Callback)**
La fonction exécutée quand l'événement survient.

Avantage majeur :

On peut attacher **plusieurs écouteurs** au même événement sur le même élément, contrairement à `onclick`.

```
events.js

// 1. Sélectionner l'élément
const button=document.querySelector('button');

// 2. Définir la fonction (optionnel)
function handleClick(event) {
  console.log('Bouton cliqué !');
}

// 3. Attacher l'écouteur
button.addEventListener('click', handleClick);

// OU avec une fonction fléchée directe
button.addEventListener('click', (e) => {
  console.log('Aussi exécuté !');
});
```

5.4 L'objet Event

Lorsque l'événement survient, le navigateur crée un objet un objet **event** et le passe automatiquement en argument à votre fonction.

.type	Type d'événement Ex: "click", "submit", "keydown"
.target	La Cible L'élément précis qui a été cliqué
.currentTarget	L'Écouteur L'élément qui écoute
.timeStamp	Le Moment Temps en ms depuis l'ouverture de la page

À retenir : L'objet `event` contient absolument toutes les informations contextuelles sur ce qui vient de se passer.

```
script.js

button.addEventListener('click', (event) => {
  // 1. "click"
  console.log(event.type);

  // 2. L'élément cliqué
  console.log(event.target);

  // 3. L'élément qui écoute
  console.log(event.currentTarget);

  // 4. Moment du clic
  console.log(event.timeStamp);
});
```

5.5 Exemple Complet : Compteur

Le cycle de vie d'une interaction : **Action** → **Donnée**
Donnée → **Affichage**.

1 L'État (State)

On stocke la donnée changeante dans une variable (`count`).

2 L'Action

L'utilisateur clique, l'événement déclenche notre fonction.

3 La Mise à Jour (Render)

On modifie la variable ET on met à jour le texte du DOM.

Concept Clé : Modifier la variable `count` ne suffit pas. Le DOM ne se met pas à jour tout seul (contrairement aux frameworks).

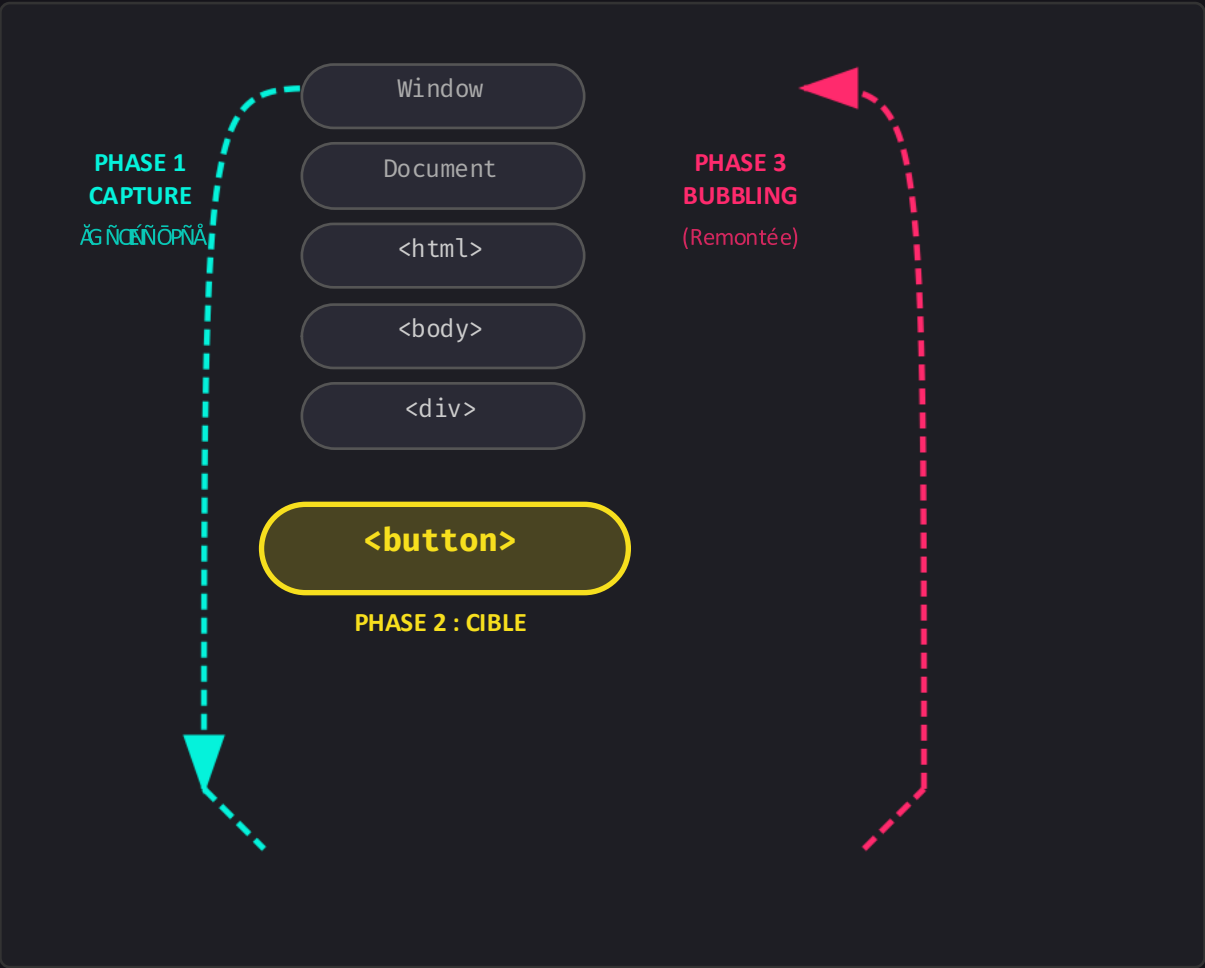
```
counter.js

let count=0;
const button=document.querySelector('button');
const display=document.querySelector('span');

button.addEventListener('click', (event) => {
  // 1. Modifier la donnée
  count++;

  // 2. Mettre à jour l'affichage
  display.textContent=count;
});
```

6.1 Les 3 Phases d'un Événement



Le Cycle de Vie d'un Événement

- 1 Capture Phase**
L'événement part de window et descend vers la cible.
- 2 Target Phase**
L'événement arrive à la cible.
- 3 Bubbling Phase**
L'événement remonte vers window (c'est ici qu'on écoute généralement).

PAR DÉFAUT :
addEventListener écoute uniquement en phase de Bubbling.

```
// Pour écouter en capture (rare) :  
element.addEventListener('click', fn, { capture: true });
```

6.2 Bubbling : L'événement Remonte

Le **Bubbling** (bouillonnement) est le comportement par défaut : l'événement remonte de l'enfant vers la racine.

- 1 **Déclenchement Initial**
L'utilisateur clique sur le <button> (la cible).
- 2 **Remontée Verticale**
L'événement traverse <div>, <body>, <html>.
- 3 **Conséquence**
Les écouteurs des parents s'activent aussi !

Attention : Si vous ne le gérez pas, un clic peut déclencher des actions indésirables sur les conteneurs parents.

```
bubbling.js

// HTML: <div class="container"><button>Clique</button></div>

const btn=document.querySelector('button');
const box=document.querySelector('.container');

// 1. Écouteur sur l'enfant (Bouton)
btn.addEventListener('click', () => {
  console.log('1. Clic sur bouton');
});

// 2. Écouteur sur le parent (Container)
box.addEventListener('click', () => {
  console.log('2. Clic détecté sur container !');
});

// RÉSULTAT CONSOLE :
> "1. Clic sur bouton"
> "2. Clic détecté sur container !"
```

6.3 Stopper la propagation

Parfois, le **Bubbling** est indésirable. La méthode `stopPropagation()` permet d'interrompre immédiatement la remontée de l'événement.

1 Isolement

L'événement ne dépassera pas l'élément actuel.

2 Silence radio

Les parents (container, body, etc.) ne seront pas notifiés.

3 Cas d'usage

Bouton " Like " dans une carte cliquable entière.

Attention :

N'abusez pas de `stopPropagation`. Cela peut "tuer" des fonctionnalités globales comme les statistiques d'analyse de clics.

```
propagation.js

const button=document.querySelector('button');
const container=document.querySelector('.container');

// 1. Écouteur sur l'enfant (Bouton)
button.addEventListener('click', (event) => {
  // 🛑 ARRÊT D'URGENCE
  event.stopPropagation();
  console.log('Bouton cliqué');
});

// 2. Écouteur sur le parent
container.addEventListener('click', () => {
  // ❌ Ne s'exécutera PAS si clic sur btn
  console.log('Container cliqué');
});
```

6.4 Annuler le comportement par défaut

De nombreux éléments HTML ont un **comportement natif** automatique (recharger la page, page, suivre un lien). La méthode `preventDefault()` permet de le bloquer.

- 1 **Liens (<a>)**
Empêche le navigateur de changer d'URL.
- 2 **Formulaires (<form>)**
Empêche le rechargement de la page lors du submit.
- 3 **Raccourcis & Menu**
Empêche le menu clic-droit (contextmenu) ou touches spéciales.

Important :

C'est la base des SPA (Single Page Applications) comme React ou Vue : on intercepte le clic pour gérer la navigation en JS sans recharger.

```
navigation.js

const link=document.querySelector('a');

link.addEventListener('click', (event) => {
  // 1. Bloque la navigation native vers href
  event.preventDefault();

  console.log('Navigation stoppée !');

  // 2. Exécute votre propre logique
  openModal();
});

// Cas du formulaire :
form.addEventListener('submit', (e) => {
  e.preventDefault(); // Pas de rechargement
});
```

6.5 target vs currentTarget

Une distinction fondamentale pour comprendre comment les événements traversent le DOM.

i event.target

C'est l'élément **déclencheur** (le plus profond). C'est celui sur lequel l'utilisateur a physiquement cliqué.

c event.currentTarget

C'est l'élément **observateur**. Celui qui possède l'écouteur (`addEventListener`) en train de s'exécuter.

À retenir :

`target` ne change jamais pendant la remontée (bubbling).
`currentTarget` change à chaque parent traversé.

```
events.js

// HTML structure :
// <div class="container">
//   <button>Clique</button>
// </div>

const container=document.querySelector('.container');

container.addEventListener('click', (event) => {
  // 1. Ce qui est réellement cliqué
  console.log(event.target);
  // <button> (si on clique le bouton)

  // 2. Qui possède l'écouteur
  console.log(event.currentTarget);
  // <div class="container"> (toujours)
});
```

6.6 Délégation d'Événements

Au lieu d'attacher 100 écouteurs sur 100 boutons, on en on en attache **un seul sur le parent** commun.

1 Moins de fonctions en mémoire, moins de liaisons au DOM.

Moins de fonctions en mémoire, moins de liaisons au DOM.

2 Éléments Dynamiques

Fonctionne automatiquement pour les éléments ajoutés *après* le chargement.

3 Maintenance Simplifiée

Toute la logique de gestion est centralisée à un seul endroit.

👉 Cela fonctionne grâce à la phase de *Bubbling*.

```
delegation.js

// HTML: <div class="container"><button>...</div>
const container=document.querySelector('.container');

// ✅ BON - Un seul écouteur parent
container.addEventListener('click', (event) => {

  // 1. Qui a réellement été cliqué ?
  const target=event.target;

  // 2. Filtrer : est-ce un bouton ?
  if (target.tagName==='BUTTON') {
    console.log('Bouton cliqué :', target.textContent);
  }
});
```

6.7 Délégation avec closest()

La méthode `.closest(selector)` est l'outil le plus robuste pour gérer la délégation d'événements.

1 Remonte l'arbre DOM

Cherche l'ancêtre le plus proche (ou l'élément lui-même) correspondant au sélecteur CSS.

2 Gère l'imbrication

Fonctionne même si on clique sur une balise enfant (ex: `<span>` dans `<span>` dans un bouton).

🔗 `closest()` remonte l'arbre DOM

🔗 `tagName` échoue si vous cliquez sur une icône à l'intérieur du bouton. `closest()` remonte jusqu'au bouton.

```
delegation-closest.js

const container=document.querySelector('.container');

container.addEventListener('click', (event) => {
  // 1. Trouver le bouton ancêtre
  const button=event.target.closest('button');

  // 2. Vérifier si on a trouvé un bouton
  if (!button) return;

  // 3. (Optionnel) Vérifier limites container
  if (!container.contains(button)) return;

  // 4. Action !
  console.log('Bouton ID :', button.dataset.id);
});
```

6.8 Cas Réel : Liste de Tâches

Gérer une liste où des éléments sont **ajoutés et supprimés** dynamiquement est le cas d'école d'école parfait pour la délégation.

1 Structure HTML

Une liste `<ul>` contient des items `<li>` avec des boutons de suppression.

2 Ciblage robuste

`closest('button')` gère le cas où on clique sur une icône *dans* le bouton.

3 Suppression du DOM

On remonte au parent `<li>` pour supprimer toute la ligne.

Avantage majeur :

Ce code fonctionne immédiatement pour toutes les nouvelles tâches ajoutées après le chargement de la page !

```
app.js

// HTML: <ul class="task-list">...</ul>

const taskList=document.querySelector('.task-list');

taskList.addEventListener('click', (event) => {
  // 1. Trouver le bouton (même si clic icône)
  const button=event.target.closest('button');

  // 2. Sécurité : vérifier si c'est bien un bouton
  if (!button) return;

  // 3. Logique métier
  const taskId=button.dataset.id;
  deleteTask(taskId); // fonction fictive

  // 4. Suppression visuelle (remonte au <li>)
  button.closest('li').remove();
});
```

7.1 Lire les valeurs des champs

Pour récupérer les données saisies par l'utilisateur, l'utilisateur, la propriété **.value** est l'outil standard.

- 1 **Champs de texte**
`<input type="text/email/...">` retourne le texte saisi.
- 2 **Zones de texte**
`<textarea>` retourne tout le texte, sauts de ligne inclus.
- 3 **Listes déroulantes**
`<select>` retourne l'attribut *value* de l'option choisie.

Í ŎPÑ Ő ŐŎŎPŎPÑ È

I MÓÓÓÓPÚ .value ÓÑÓÓÑ PÓP ÓP ÓÑ NÓMÓÑÑÑ NÓMÓPÚÓÑCE
ÁPÓÑÁP ÚÓ Ñ ÓÓP ÓP Ó <input type="number">B

```
form-data.js

// 1. Sélectionner les éléments
const input=document.querySelector('input[type="email"]');
const textarea=document.querySelector('textarea');
const select=document.querySelector('select');

// 2. Lire les valeurs (au moment de l'exécution)
console.log(input.value);
// → "jean.dupont@exemple.fr"

console.log(textarea.value);
// → "Bonjour, j'aimerais..."

console.log(select.value);
// → "option-2"
```

7.2 input : retour en temps réel

L'événement **input** est le standard moderne pour réagir aux modifications d'un champ.

- 1 Déclenchement immédiat**
S'active à chaque frappe, suppression ou modification.
- 2 Capture tout type de saisie**
Clavier, souris (copier/coller), dictée vocale, autocomplétion.
- Ⓒ Expérience utilisateur (UX)**
Permet le feedback visuel instantané (recherche, validation).

Comparaison : Contrairement à `change` qui attend la perte du focus (clic ailleurs), `input` est continu.

```
live-feedback.js

// HTML: <input type="text"> <div class="feedback">
const input=document.querySelector('input');
const fb=document.querySelector('.feedback');

// Écoute chaque changement en temps réel
input.addEventListener('input', (event) => {
  // Récupérer la valeur actuelle
  const value=event.target.value;

  // Mise à jour de l'affichage
  fb.textContent=`Vous avez saisi : ${value}`;

  // Exemple de validation simple
  if (value.length>10) {
    input.style.borderColor='red';
  } else {
    input.style.borderColor='#333';
  }
});
```

7.3 Événement change : Après Validation

Contrairement à `input`, l'événement `change` ne déclenche que lorsque la modification est l'utilisateur.

- 1 Champs texte (input)**
Se déclenche à la perte du focus (blur) si la valeur a changé.
- 2 Listes (select) / Checkbox**
Se déclenche immédiatement après le clic de sélection.
- 🕒 Usage recommandé**
Validation finale, appels API (lourds), mise à jour de filtres.

Différence Clé :

`input` = temps réel (chaque frappe).
`change` = état final (validation).

```
validation.js

// Sélection d'un menu déroulant
const select=document.querySelector('select');

// Se déclenche après le choix d'une option
select.addEventListener('change', (event) => {
  const valeur=event.target.value;

  console.log('Nouvelle sélection :', valeur);

  // Exemple : Filtrer une liste de produits
  updateProductList(valeur);
});
```

7.4 Événements focus et blur

Ces deux événements permettent de savoir quand quand l'utilisateur **entre** ou **sort** d'un champ de formulaire.

1 focus (Entrée)

Déclenché quand l'élément est sélectionné (clic ou Tabulation).

2 blur (Sortie)

Déclenché quand l'élément perd la sélection (clic ailleurs).

3 Usage UX

Indispensable pour l'accessibilité, les aides visuelles et la validation.

Différence avec change :

L'événement `blur` se déclenche dès la sortie, même si la valeur n'a pas été modifiée.

```
ux-interaction.js

const input=document.querySelector('input');

// 1. Gain de focus : mise en valeur
input.addEventListener('focus', () => {
  console.log('Saisie active');
  input.style.borderColor='#3b82f6'; // Bleu
  input.style.backgroundColor='#1e1e28';
});

// 2. Perte de focus : retour à la normale
input.addEventListener('blur', () => {
  console.log('Saisie terminée');
  input.style.borderColor='#666'; // Gris
  input.style.backgroundColor='';
});
```

7.5 Soumettre un formulaire

Pour gérer un formulaire en JavaScript (AJAX/SPA), (AJAX/SPA), il est crucial d'**intercepter** la soumission soumission standard.

1 Écouter le formulaire

L'événement `submit` se place sur la balise `<form>`, pas sur le bouton.

2 Empêcher le rechargement

`event.preventDefault()` est obligatoire pour ne pas rafraîchir rafraîchir la page.

3 Récupérer les données

Accédez aux champs via `form.querySelector`.

Important : `submit` détecte aussi la touche "Entrée" dans un champ, contrairement à un simple `click` sur le bouton.

```
form-handler.js

const form=document.querySelector('form');

form.addEventListener('submit', (event) => {
  // 1. Empêcher l'envoi par défaut (rechargement)
  event.preventDefault();

  // 2. Récupérer les valeurs
  const email=form.querySelector('input[name="email"]').value;
  const pwd=form.querySelector('input[name="pwd"]').value;

  console.log('Email :', email);
  console.log('Password :', pwd);

  // 3. Valider / envoyer les données (AJAX)
});
```

7.6 Validation Simple

Validez les données **côté client** pour offrir un retour retour immédiat à l'utilisateur avant de solliciter le solliciter le serveur.

1 Interceptor

Utiliser `submit + preventDefault()`.

2 Vérifier

Tester la validité (email, longueur, regex).

3 Informer

Afficher un message d'erreur clair.

Important :

La validation JS est pour le confort (UX). La validation de sécurité doit **toujours** être faite côté serveur également.

```
validation.js

const form=document.querySelector('form');
const emailInput=form.querySelector('#email');
const errorMsg=form.querySelector('.error');

form.addEventListener('submit', (event) => {
  // 1. Bloquer l'envoi par défaut
  event.preventDefault();

  // 2. Vérifier la valeur
  if (!emailInput.value.includes('@')) {

    errorMsg.textContent='Email invalide !';
    emailInput.classList.add('invalid');
    return; // Arrêter ici
  }

  // 3. Envoyer si tout est OK
  console.log('Validation OK, envoi...');
  // form.submit();
});
```

7.7 FormData : Tout récupérer

L'interface `FormData` capture automatiquement tous les champs d'un formulaire pour simplifier leur traitement.

1 Capture Globale

Évite de sélectionner chaque `input` manuellement par son ID.

2 Types Complexes

Gère automatiquement les fichiers, les cases à cocher et les boutons radio.

3 Prêt pour l'envoi

Le format est compatible directement avec `fetch()`.

Important :

Pour que `FormData` fonctionne, chaque champ du formulaire doit impérativement avoir un attribut `name="..."`.

```
form-handler.js

// 1. Sélection du formulaire
const form=document.querySelector('form');

form.addEventListener('submit', (e) => {
  e.preventDefault();

  // 2. Création de l'objet FormData
  const formData=newFormData(form);

  // 3. Récupère valeurs via attribut "name"
  const user=formData.get('username');
  const mail=formData.get('email');

  console.log({ user, mail });
});
```

8.1 Séparer HTML et JavaScript

Mélanger le comportement (JS) et la structure (HTML) est une **dette technique** qui rend le code difficile à maintenir.

- ✗ **Jamais d'attributs "on..."**
onclick, onmouseover polluent le HTML.
- ✓ **Séparation des responsabilités**
Le JS reste dans les fichiers .js.
- ! **Maintenabilité**
Permet de modifier le comportement sans toucher au balisage.

Règle d'or :

Toujours utiliser `addEventListener()` pour attacher des événements, jamais via le HTML directement.

```
index.html & script.js

// ✗ MAUVAIS - Mélange des genres
<!-- Dans le fichier HTML -->
<button onclick="doSomething()"> Cliquez </button>

// ✓ BON - Séparation propre
<!-- HTML : juste la structure -->
<button class="my-button"> Cliquez </button>

// JavaScript : tout le comportement
const btn=document.querySelector('.my-button');
btn.addEventListener('click', () => {
  console.log('Cliqué !');
});
```

8.2 Limiter les Manipulations

Chaque modification du DOM force le navigateur à recalculer la mise en page (**Reflow**) et à redessiner (**Repaint**).



Boucle Directe

100 éléments = 100 calculs



DocumentFragment

100 éléments = 1 seul calcul

Le Fragment est un conteneur DOM virtuel "hors-écran".

optimization.js

// ✗ LENT : Touche le DOM à chaque tour

```
items.forEach(item=> {  
  const div=document.createElement('div');  
  div.textContent=item;  
  document.body.appendChild(div); // 🐢 Reflow!  
});
```

// ✓ RAPIDE : Groupe les insertions

```
// Crée un conteneur hors du DOM  
const fragment=document.createDocumentFragment();  
  
items.forEach(item=> {  
  const div=document.createElement('div');  
  
  div.textContent=item;  
  fragment.appendChild(div); // 👍 En mémoire  
});  
  
// Insère tout en une seule fois  
document.body.appendChild(fragment);
```

8.3 Nettoyer les Écouteurs

Les écouteurs d'événements persistants peuvent causer des **fuites de mémoire**. Il est crucial de les retirer quand ils ne sont plus nécessaires.

1 Fonctions nommées obligatoires

Impossible de retirer une fonction anonyme `() => {}` car on n'a pas sa référence.

2 removeEventListener

`removeEventListener` nécessite exactement les mêmes arguments que `add`.

3 Avant suppression

Nettoyez toujours les écouteurs avant de supprimer un élément du DOM.

Important : Si l'élément disparaît, l'écouteur peut rester attaché en mémoire s'il n'est pas nettoyé explicitement.

```
cleanup.js

const button=document.querySelector('button');

// 1. Définir une fonction nommée (référence)
const handleClick= () => {
  console.log('Cliqué');
};

// 2. Ajouter l'écouteur
button.addEventListener('click', handleClick);

// 3. Supprimer plus tard (avec la même réf)
button.removeEventListener('click', handleClick);

// ❌ Ceci ne marcherait PAS :
// button.removeEventListener('click', () => {...})
```

8.4 Organiser le code (classe)

Pour éviter le "code spaghetti", regroupez la logique, les données, les événements dans des **classes** ou des modules.

1 Constructeur

Sélection des éléments DOM et stockage.

2 Initialisation (init)

Mise en place des écouteurs d'événements.

3 Méthodes dédiées

Logique métier séparée (ex: deleteTask).

Avantage : Meilleure lisibilité, maintenance simplifiée et facilité de test.

```
task-manager.js

class TaskManager {
  constructor() {
    this.list = document.querySelector('.task-list');
    this.init();
  }

  init() {
    this.list.addEventListener('click', (e) => this.handleClick(e));
  }

  handleClick(event) {
    const button = event.target.closest('button');
    if (button) this.deleteTask(button);
  }

  deleteTask(button) {
    button.closest('li').remove();
  }
}

// Utilisation
new TaskManager();
```

8.5 Événement DOMContentLoaded

Si votre script JavaScript est chargé dans le **<head>**, il s'exécute avant que le corps de la page page n'existe.



Le risque

`querySelector` renvoie `null` car l'élément HTML n'a pas encore été analysé par le navigateur.



La solution

Attendre l'événement `DOMContentLoaded` qui signale que l'arbre DOM est complet.



Alternative moderne

Utiliser l'attribut `<script defer>` qui diffère l'exécution automatiquement.

Règle d'or :

Ne jamais manipuler le DOM avant d'être sûr qu'il est chargé.



index.js

```
// ✗ RISQUÉ (si dans <head>)
// Le bouton n'existe pas encore !
document.querySelector('button');
// → null (crash possible)

// ✔ SÛR
document.addEventListener('DOMContentLoaded', () => {
  // Ici, le HTML est prêt
  const btn=document.querySelector('button');

  btn.addEventListener('click', () => {
    console.log('Prêt !');
  });
});
```

9.1 Ce Qu'on Vient d'Apprendre



Manipuler le DOM

Créer, modifier, insérer et supprimer des éléments HTML dynamiquement.



Sélection et Navigation

Maîtriser `querySelector` et se déplacer dans l'arbre (parents/enfants).



Gestion des Événements

Comprendre `addEventListener`, l'objet `Event` et la propagation.



À L'ŒUVRE

Délégation d'événements, gestion des formulaires et sécurité (XSS).



NIVEAU VALIDÉ

DOM API

Event Loop

Listeners

Forms

Performance

Debugging

9.2 Limites du DOM impératif

L'approche "Vanilla JS" fonctionne bien pour de de petits scripts, mais devient **difficile à maintenir** quand l'application grandit.

1 État (variables) et Affichage (DOM) sont séparés.

2 Approche Impérative
On doit décrire *comment* faire chaque modification.

3 Risque d'Erreurs
Oublier une mise à jour crée des bugs d'interface.

Plus de changements, plus de bugs

Plus il y a de changements, plus il est difficile de garder l' affichage synchronisé avec les données.

```
counter-vanilla.js

// État (données) de l'application
let count=0;

const btn=document.querySelector('button');
const span=document.querySelector('span');

btn.addEventListener('click', () => {
  // 1. Modifier les données
  count++;

  // 2. Mettre à jour l'affichage MANUELLEMENT
  span.textContent=count;

  // Si ce nombre est affiché ailleurs,
  // il faut penser à le mettre à jour aussi !
});
```

9.3 Impératif vs Déclaratif

IMPÉRATIF (Vanilla JS)

Vanilla

Focus : "COMMENT faire"

- ✗ Manipuler le DOM étape par étape
- ✗ Sélectionner manuellement (querySelector)
- ✗ Synchroniser l'état et l'UI soi-même
- ✗ Difficile à maintenir quand ça grandit

VS

DÉCLARATIF (Frameworks)

React/Vue

Focus : "QUOI afficher"

- ✓ Décrire l'état final désiré
- ✓ Le Framework manipule le DOM pour vous
- ✓ Synchronisation automatique (State → UI)
- ✓ Code prévisible et lisible

★ LISIBILITÉ

★ MAINTENANCE

★ TESTABILITÉ

9.4 Aperçu React : Le Compteur

En React, on ne manipule plus le DOM manuellement. On définit un **état** et l'interface s'adapte automatiquement.

- 1 **État (State)**
Une variable `count` qui déclenche le rendu.
- 2 **Génération d'HTML**
On décrit le HTML final voulu, mélangé au JS.
- 3 **Réactivité**
Changer l'état → React met à jour le DOM.

Différence majeure :

Plus besoin de `document.querySelector` ni de gérer manuellement les mises à jour avec `textContent` !

```
Counter.jsx

import { useState } from 'react';

export function Counter() {
  // 1. Déclaration de l'état (count = 0)
  const [count, setCount] = useState(0);

  return (
    <div>
      /* 2. Affichage automatique */
      Clics : {count}

      <button onClick={() => setCount(count + 1)}>
        Incrémenter
      </button>
    </div>
  );
}
```

9.5 Vous savez faire maintenant



Créer des interfaces riches et dynamiques en maîtrisant Vanilla JS.



Logique des frameworks

Comprendre ce que React ou Vue abstraient (événements, state, rendu).



Débogage d'événements

Identifier et résoudre les problèmes de propagation, bubbling et contexte.



Performance et Maintenance

Optimiser les manipulations DOM et écrire du code JS modulaire.



PRÊT POUR REACT

Vanilla JS

Mastery

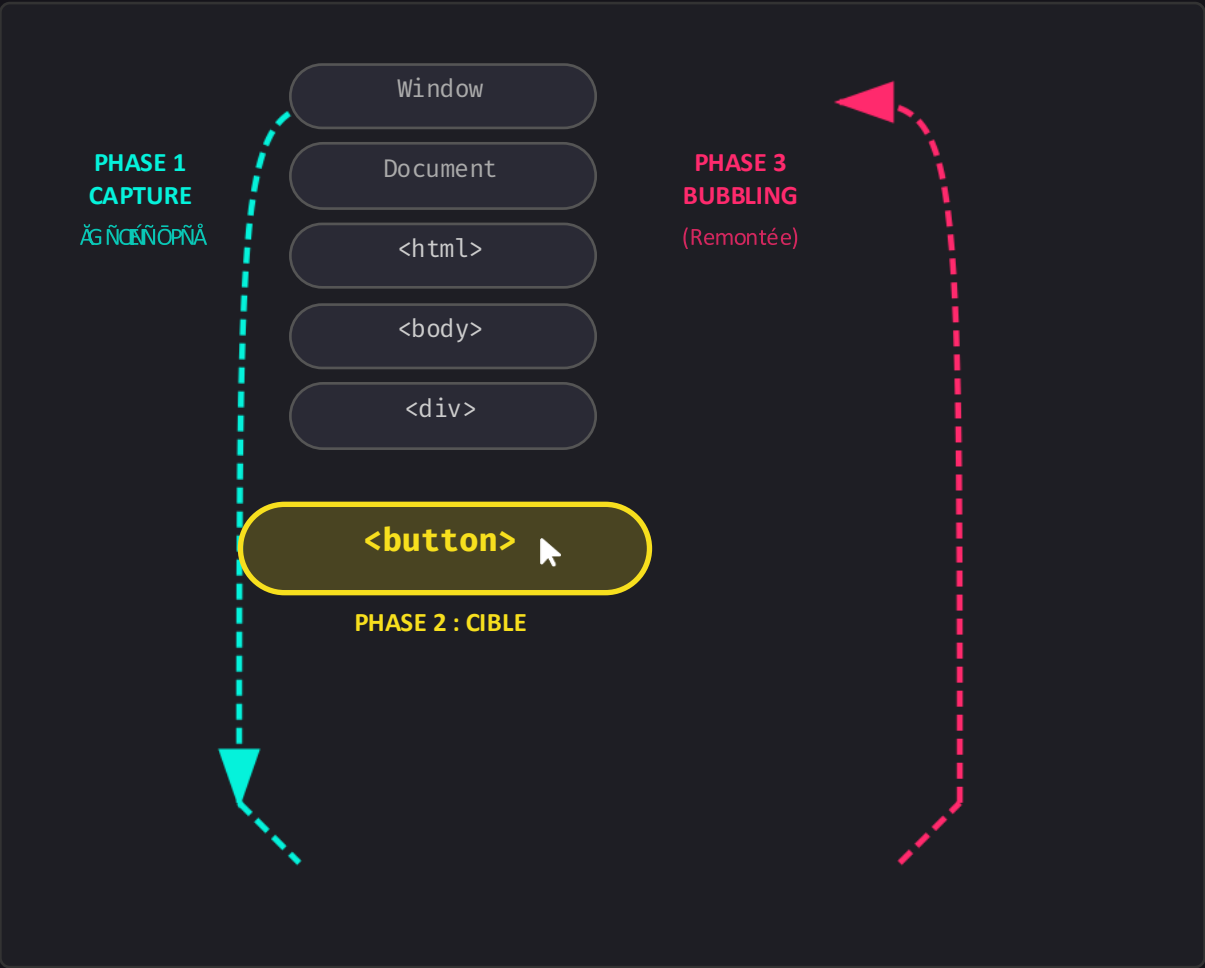
Clean Code

Debugging

Architecture

Optimisation

Visuel : Flux d'un Événement



Résumé du Cycle d'Événement

- 1 Capture (Descente)**
Déclenchement : L'utilisateur clique. Le signal part de la racine et descend jusqu'à la cible.
- 2 Cible (Target)**
L'événement arrive à la cible (le bouton).
- 3 Bubbling (Remontée)**
L'événement remonte vers les parents. C'est ici que la délégation d'événements opère.

**À RETENIR :**
Les handlers sont exécutés selon la phase écoutée. La propagation peut être stoppée via `stopPropagation()`.

```
// Par défaut (Bubbling) :  
btn.addEventListener('click', handler);  
// En Capture : btn.addEventListener(..., {capture: true});
```

THE END

COURS DOM & ÉVÉNEMENTS

Prêt pour React !!!